네 대의 Arduino로 함께 생성하는 작은 문장

프로젝트 계획서 · 2026.10.07 · 발표 2026.10.20

작은 영어 문자 신경망을 Arduino Uno R3 네 대에 나누고, UART·I2C·SPI로 부분 계산 결과를 모아 짧은 문장을 생성한다. PC는 학습·업로드·출력 확인에만 사용한다. 실행 중 다음 문자 예측은 보드가 직접 수행한다.

계획의 중심 질문

한 대에서 동작하는 동일한 신경망을 여러 보드가 협력하여 계산할 수 있는가? 세 통신 방식의 실제 전송을 같은 추론 작업 안에서 겹치게 만들 수 있는가? 성능과 메모리의 변화는 측정값으로 비교한다.

구분	현재 확보한 증거	이번 프로젝트에서 확인할 것
1보드	Uno 타깃 컴파일, 호스트 추론·펌웨어 mock 검증	실물 출력·시간·스택
4보드	설계 단계	분산 로짓 일치, 세 버스 동시 전송
모델 확장	학습 목표만 설정	용량·속도·미학습 문장 품질

과제 원문

“Using 4 arduinos, create a cooperating application that utilize all 3 communication methods. (UART, SPI, I2C) At least 1 arduino uno should run all three protocols at the same time. The reports can be the same for the teams. However, individual reports should contribution (yes/no) for each of the members (excluding yourself) on the first page. Prepare for presentation on Oct 20. The presentation must be under 5 minutes.”

성공의 정의

네 보드가 한 모델의 계산을 협력 수행하고, 메인 Uno에서 UART·I2C·SPI 실제 전송이 겹치는 파형을 확보한다. 현재 작은 모델의 분산 결과를 먼저 검증한 뒤 확장을 시도한다. 확장 실패 시 작은 분산 모델로 발표하되 동시 통신 요건은 반드시 충족한다.

이 문서는 완료 보고서가 아닌 계획서이다. 개인·팀 정보는 포함하지 않는다. 최종 개인 보고서 첫 페이지에는 본인을 제외한 각 팀원의 기여 여부 Yes/No를 작성해야 한다.

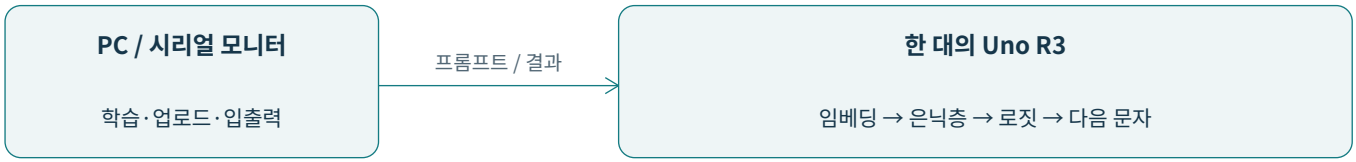
과제 요구사항을 구현으로 연결

역할과 증거

과제 요구	설계 대응	제출 증거
Arduino 4대 협력	메인 1대 + 신경망 계산 워커 3대	배선 사진·각 보드 빌드·추론 로그
UART·SPI·I2C 활용	워커마다 서로 다른 통신 링크	프레임 로그·오류 주입 시험
한 Uno에서 세 프로토콜 동시에	메인이 세 인터페이스를 유지하고 전송 중첩	동일 job ID의 로직 분석기 파형
발표 5분 미만	4분 40초 발표 구성	리허설 타이머·최종 영상
개인 보고서의 기여 표시	최종 보고서 첫 페이지에 타인 Yes/No	개인별 작성 확인

응용 시나리오

사용자가 hello, the cat, a robot 같은 짧은 영어 시작 문구를 보낸다. 메인이 최근 16문자를 세 워커로 보내고, 워커가 각자 맡은 은닉 뉴런의 출력을 계산한다. 메인이 29개 문자 후보의 점수를 합쳐 다음 문자를 선택한다. 줄 끝 토큰 또는 최대 출력 길이까지 반복한다.



추론 산술은 Uno 내부에서 수행 · PC는 표시만 담당

한 대와 네 대 비교의 원칙

작은 동일 모델·동일 문맥·동일 선택 규칙을 먼저 고정한다. 분산 통신의 비용만 분리하여 측정하고, 이후 더 큰 모델의 가능성을 별도로 평가한다. 워커는 세 대이므로 계산 자원이 네 배가 되는 것은 아니다. 보드 메모리도 하나의 연속 공간으로 합쳐지지 않는다.

UART→I2C→SPI를 순서대로 한 번씩 실행한 기록은 동시 실행의 증거가 아니다. 계산이 겹친 것과 통신 신호가 겹친 것도 구분한다. 세 통신 전송의 실제 시간 구간이 중첩되어야 한다.

한 대의 Uno: 출발점과 제한

현재 신경망

항목	현재 값
구조	문자 임베딩 → 평탄화 → ReLU 은닉층 → 문자 로짓
입력·출력	29문자, 최근 문맥 16문자, 임베딩 4차원
은닉층	48뉴런, greedy 다음 문자 선택
모델 데이터	가중치·bias 4,888 B, int8 가중치 + float32 bias·scale
Uno 빌드	Flash 8,614 / 32,256 B · 전역 SRAM 462 / 2,048 B
코퍼스	17줄·627문자, 서로 다른 단어 40개·문장 30개
학습 지표	학습 데이터 정확도 약 96.0%; 일반화 지표 아님

확인된 출력 예시

hello → hello friend. we can learn together.

the cat → the cat is small. the cat likes milk.

a robot → a robot can help you.

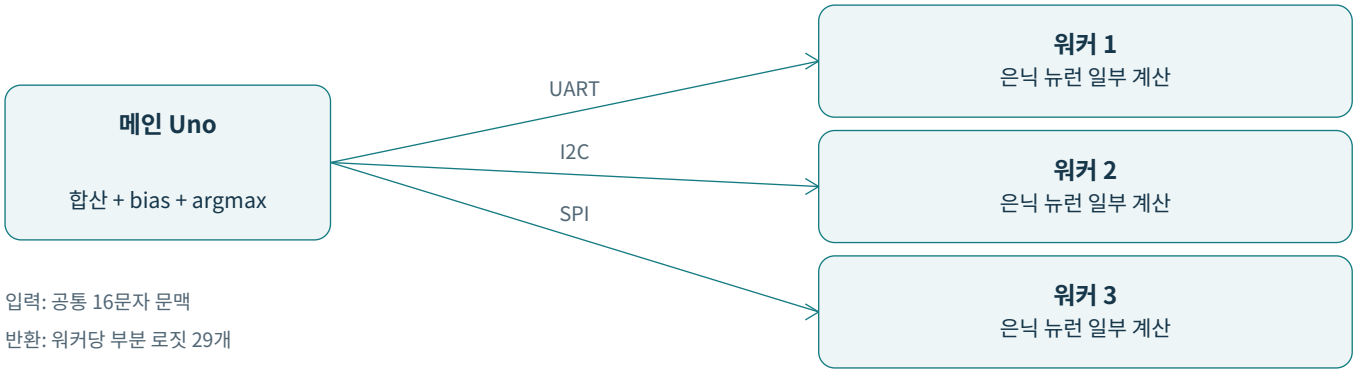
이 예시는 호스트 C++와 Python 참조에서 일치했다. 보드가 연결되지 않아 실물의 출력·속도·최대 스택은 아직 검증하지 않았다. 남은 SRAM 1,586 B는 정적 빌드 수치이며 실행 중 안전성을 계측한 보장이 아니다.

범위와 재현성

작은 문자 신경망이며 Transformer, SmolLM2, 지식 증류 모델이 아니다. 직접 작성한 영어 문장을 학습한다. SmolLM2-135M의 1바이트/파라미터 가중치만 잡아도 약 135 MB로 Uno 용량을 크게 초과하므로 대상에서 제외한다 [1, 2]. 현재 데모는 범용 대화나 한국어 모델을 목표로 하지 않는다.

공유 추론 헤더 tiny_model.h, 학습·export 도구 train.py, 편집 가능한 corpus.txt 및 검증 로그를 보관한다. Flash의 PROGMEM에서 가중치를 읽고 보드가 직접 계산한다. PC에서 계산한 토큰을 보드로 보내는 방식은 허용하지 않는다.

네 대의 Uno: 하나의 신경망을 분할



입력: 공통 16문자 문맥

반환: 워커당 부분 로짓 29개

정확한 합산 규칙

워커 k는 임베딩 E를 복제하고, 자기 은닉 뉴런의 입력 가중치 Wk, bias bk, 출력 가중치 Vk만 저장한다. 모든 워커는 같은 16문자 문맥을 받는다.

hk = ReLU(flatten(E[context]) · Wk + bk)

pk = hk · Vk → 각 워커가 29개 float32 부분 로짓(116 B)을 반환

최종 logits = p1 + p2 + p3 + 출력 bias c

메인은 출력 bias를 한 번만 더하고 argmax를 수행한다. 워커별 최상위 문자만 보내면 전역 최상위 문자를 복원할 수 없으므로 반드시 전체 부분 로짓을 보낸다.

단계	워커당 은닉층	총 은닉층	검증 목적
첫 분산 모델	16	48	기존 1보드와 같은 가중치의 수치 동등성
확장 후보	192~240	576~720	Flash 활용·생성 품질·속도 절충

용량 목표와 해석

워커 모델 저장량 근사는 (16×4 + 29 + 4)×h + 29×4 = 97h + 116 B이다. scale·헤더·코드 등은 별도다. 출력 bias 116 B는 메인에 둔다. 확장 가중치 전체는 약 55~70 KB이며 현재 4.9 KB 모델 대비 약 11~14배, 최대 용량을 활용한 1보드 대비 대략 3배의 가능성이다. 구현 후 링크 결과로 다시 확인한다.

1차 학습 목표는 제한된 로봇·동물 영역의 단어 150개·짧은 문장 300개이다. 탐색 범위는 단어 100~200개·문장 200~400개, 확장 후보는 단어 200~400개·문장 400~800개이다. 모두 미검증 학습 목표이며 단어·문장 수의 하드웨어상 최대치나 품질 보장은 아니다. 큰 모델은 현재 작은 모델보다 느려질 수 있다.

배선·패킷·동시 전송의 구현

링크 및 핀 계획

링크	메인 Uno	해당 워커 Uno	초기 설정
PC 모니터	D0 RX / D1 TX 하드웨어 UART	USB 연결은 메인만 입출력용	9600 baud
워커 1 UART	D2 RX / D3 TX	D1 TX → D2, D0 RX ← D3	9600, 8N1
워커 2 I2C	A4 SDA / A5 SCL	같은 핀, 주소 0x12	100 kHz, pull-up
워커 3 SPI	D10 SS, D11 MOSI, D12 MISO, D13 SCK	같은 핀, SS D10 INPUT	SCK 125 kHz

메인 D10은 OUTPUT이다. 5 V Uno끼리 공통 GND를 연결한다. 개별 USB로 전원을 공급한다면 독립된 5 V 전원 레일을 서로 묶지 않는다. UART 워커는 업로드 시 D0/D1 외부 연결을 분리하고, 동작 중 USB 시리얼 모니터를 사용하지 않는다. SPI 보드의 D13은 SCK이므로 활동 LED로 쓰지 않고 필요하면 D4를 디버그 핀으로 사용한다.

프레임과 상태 머신

프레임은 job ID, op, seq, 길이, payload, CRC8을 포함한다. 문맥 16 B를 보내고 ready 상태를 확인한 뒤 116 B 결과를 읽는다. Wire의 기본 버퍼는 32 B이므로 payload 최대 24 B와 헤더 최대 8 B로 나누고, 청크 ACK·타임아웃·작업 ID로 누락·오래된 결과를 검사한다 [3]. 오류 시 실패를 표시하며 PC 추론으로 몰래 대체하지 않는다.

메인은 send → start → ready 확인 → result 수집 → aggregate 상태 머신으로 동작한다. 워커의 I2C 콜백과 SPI ISR은 버퍼 입출력·완료 결과 제공만 수행한다. 신경망 계산은 워커 main loop에서 실행하여 ISR을 길게 점유하지 않는다.

핵심 난점: Software UART

기본 SoftwareSerial TX는 cli()로 인터럽트를 막으므로 그대로 동시 통신을 달성했다고 주장하지 않는다 [4]. 타이머 기반 비차단 software UART와 짧은 ISR, 하드웨어 I2C·SPI를 조합할 계획이다. 타이머 UART 전송 중 I2C·SPI 전송을 진행하며 비트 타이밍·바이트 손실을 확인한다. 달성되지 않으면 통신 설계를 재조정한다.

로직 분석기로 UART TX, I2C SDA/SCL, SPI SCK/MOSI/SS의 6채널을 기록한다. 필요 시 UART RX와 SPI MISO를 더해 8채널을 쓴다. 디코딩된 전송·job ID·타임스탬프를 저장하고, UART 프레임·I2C 트랜잭션·SPI 트랜잭션 세 가지가 모두 겹치는 하나의 공통 시간 구간을 제시한다. 두 링크씩만 겹치거나 계산만 겹치는 기록은 통과로 보지 않는다.

구현 1: 학습 결과를 세 워커로 내보내기

아래 파일·함수·명령 중 "예정" 표시는 앞으로 작성할 설계이다. 이번 계획서 수정에서는 펌웨어나 학습 모델을 변경하지 않는다. 기존 tiny_uno/를 비교 기준으로 보존한다.

데이터부터 나누고, 같은 가중치를 유지

corpus.txt는 29문자 어휘의 제한된 소문자 영어로 작성한다. 새 품질 평가에서는 문장을 먼저 학습/검증 세트로 나누고, 그 다음 각 세트에서 16문자 문맥을 만든다. 같은 문장의 문자 조각이 양쪽에 섞이는 누수를 막는다. 기존 96.0%는 이 검증을 거친 점수가 아니다.

현재 tools/train.py의 고정 seed·1800단계 학습을 재현한 뒤 model.npz를 동결한다. 임베딩과 두 가중치 텐서는 각각 하나의 대칭 int8 scale, bias는 float32를 사용한다. 48뉴런 모델의 배열은 e[29,4], w[64,48], o[48,29], b[48], c[29], scale se/sw/so이다.

워커	은닉 뉴런 범위	복제·분할 배열
UART	[0,16)	e 전체, w[:,0:16], b[0:16], o[0:16,:]
I2C	[16,32)	e 전체, w[:,16:32], b[16:32], o[16:32,:]
SPI	[32,48)	e 전체, w[:,32:48], b[32:48], o[32:48,:]

export_split.py 구현 예정

새 host/export_split.py는 이미 양자화된 배열을 그대로 자르고 원래 se/sw/so를 유지한다. 각 조각을 별도로 재양자화하면 동일 모델 검증 조건이 바뀌므로 하지 않는다. 출력 bias c[29]는 메인만 저장한다.

```
load frozen model.npz + metadata
for (link, lo, hi) in [(uart,0,16),(i2c,16,32),(spi,32,48)]:
    emit e, w[:,lo:hi], b[lo:hi], o[lo:hi,:], se, sw, so
    emit model ID, vocabulary ID, dimensions, shard range
emit main_bias.h: c[29] + same model/vocabulary IDs
```

산출물은 model_uart.h, model_i2c.h, model_spi.h, main_bias.h이다. 배열은 PROGMEM에 두고 int8은 pgm_read_byte, float는 pgm_read_float로 읽는다. 상수·버퍼·코드를 포함한 각 보드 링크 결과와 실행 중 스택 여유를 확인한다.

확장은 새 학습 이후에만

총 H=576 또는 720 후보는 전체 큰 네트워크를 먼저 재학습한 뒤 3등분한다. 기존 48개 뉴런의 가중치를 복제하여 큰 모델이 되었다고 주장하지 않는다. 미학습 문장 품질, 모델 ID, Flash/SRAM, 지연을 다시 평가한 뒤 발표 모델을 선택한다.

기존 실행 명령: OPENBLAS_NUM_THREADS=1 python3 tools/train.py → python3 tools/verify.py → ./tools/build.sh. 분할 후 검증 명령 python3 host/export_split.py, python3 host/test_distributed.py는 파일 구현 이후에만 실행한다.

구현 2: 메인과 워커의 상태 머신

작성 예정 모듈과 책임

예정 경로	핵심 책임
firmware/main/main.ino	start_job, poll_link, aggregate_next, 입력·출력
firmware/worker_uart/worker_uart.ino	하드웨어 UART 프레임 처리·부분 추론
firmware/worker_i2c/worker_i2c.ino	짧은 콜백·캐시된 응답·부분 추론
firmware/worker_spi/worker_spi.ino	SPI byte pipeline·부분 추론
firmware/shared/protocol.h	프레임·CRC·model/job ID·오류 정의
firmware/shared/neural_shard.h	next_partial(ctx, out29), PROGMEM 읽기
host/export_split.py / host/test_distributed.py	PC에서 분할·수치 비교, 실행 중 추론 아님

메인: 한 문자 작업의 의사코드 - 구현 예정

고정 버퍼는 context[16], totals[29] float, 링크별 청크 상태와 receivedMask를 둔다. 한 작업 동안 context는 바꾸지 않고, 결과 전체를 받은 워커만 합산한다.

```
for each next character (maximum 80):
    totals = global_bias; receivedMask = 0; jobId++
    dispatch identical context + jobId to all 3 links
    while receivedMask != 0b111:
        poll_link(UART); poll_link(I2C); poll_link(SPI)
        if complete valid result(modelId, jobId, CRC, chunks):
            if worker not yet in receivedMask:
                totals += worker.partial29; set worker bit
        if timeout/error: clear job state; print ERROR; stop
    token = ascending-index argmax(totals)
    if token == EOS(0): stop
    push(context, token); output token
```

워커: 계산은 main loop에서

IDLE → RECEIVE → COMPUTE → RESULT_READY → IDLE 순서로 상태를 둔다. 전체 문맥과 START를 검증·복사한 뒤 계산하고 29개 부분 로짓을 캐시한다. 재전송된 같은 job은 재계산·중복 합산 없이 ACK 또는 캐시된 결과로 처리한다.

ISR/콜백은 수신 바이트 큐, 상태 플래그, 이미 준비된 TX 버퍼만 처리한다. 신경망 계산이나 긴 float 변환을 넣지 않는다. 출력 bias는 워커에서 더하지 않는다. 메인은 같은 ID의 3개 완전한 결과 후에만 다음 job을 시작한다. 부동 소수점 합산 순서 차이는 abs/rel 1e-4 비교와 선택 토큰 검증으로 확인한다.

구현 3: 32바이트 프레임과 복구 규칙

공통 직렬화 규격 - 구현 예정

바이트 위치	필드	규칙
0 / 1 / 2	SOF / version / op	0xA5 / 1 / 명령 코드
3~4	jobId	uint16 little-endian
5 / 6	seq / payloadLen	청크 번호 / 유효 payload 바이트 수
7~30	payload[24]	사용하지 않는 뒤쪽은 0으로 채움
31	CRC8	poly 0x07, init/xorout 0x00, non-reflected

헤더 7 B + payload 24 B + CRC 1 B = 고정 32 B이다. CRC는 0 padding을 포함한 앞 31 B 전체를 검사한다. op는 LOAD_CONTEXT, START, STATUS, GET_RESULT, RESULT, ACK, ERROR로 정의한다. LOAD_CONTEXT의 유효 데이터는 16 B이다. 부분 로짓 116 B는 seq 0~4의 24/24/24/24/20 B 청크로 나눈다.

float는 sizeof(float)==4와 IEEE754 호환성을 확인하고 memcpy로 little-endian binary를 직렬화한다. ASCII 숫자로 바꾸거나 float 포인터를 비정렬 버퍼에 직접 캐스팅하지 않는다. 시작 STATUS에는 모델 hash 기반 ID·어휘 ID·차원·담당 은닉 범위를 넣는다. 메인은 0~47이 중복·누락 없이 한 번씩 포함되는지 검사하여 잘못된 shard 업로드를 거절한다.

링크마다 같은 규격, 다른 전송 절차

링크	송수신 절차
UART	메인은 timer UART, 워커는 hardware UART. 최소 64 B ring buffer에 쌓고 SOF 재탐색·CRC 확인. ACK/STATUS로 응답 흐름을 제한한다.
I2C	주소 0x12. GET_RESULT 수신 시 seq를 latch. onRequest는 준비된 frame 최대 32 B만 복사하고 메인이 32 B 읽어 검증한다.
SPI	SS 아래 요청 frame 전송 → 응답 준비 상태 확인 → 별도 read transaction에서 dummy 32 B로 응답을 clock. 첫 바이트 전에 SPDR를 preload한다.

SPI는 받은 현재 바이트의 응답을 같은 clock에서 즉시 돌려줄 수 없다. 첫 dummy/status 바이트의 pipeline 지연과 응답 경계를 규격에 명시하고 테스트한다. slave ISR이 SPDR를 갱신하도록 바이트 사이 간격을 둔다. Wire 32 B 제한과 맞춘 공통 frame은 메인 비차단 TWI 엔진에도 적용한다 [3].

실패를 정상 경로로 다루기

청크별 ACK·CRC·job ID를 검사한다. 워커별 5-bit chunk mask로 중복 RESULT를 건너뛰고 5청크 완료 후 워커당 한 번만 합산한다. START/GET_RESULT 재시도는 같은 job에서 멱등 처리한다. 누락·CRC 실패에는 최대 2회 재시도하고, 이전 job 결과는 버린다. 통신 timeout과 전체 작업 timeout을 분리한다. 전체 작업 10초는 초기 잠정 설정이며 계산 시간을 실측해 조정한다. 계산 완료를 30 ms 같은 미측정 값으로 단정하지 않는다.

메모리 부족·잘못된 model ID·최종 timeout이면 작업 상태를 초기화하고 ERROR를 출력한다. 다음 정상 입력은 다시 시작할 수 있어야 한다. PC의 모델 호출로 결과를 대체하지 않는다.

구현 4: 비차단 통신과 단계별 통합

메인 Uno의 하드웨어·타이머 계획

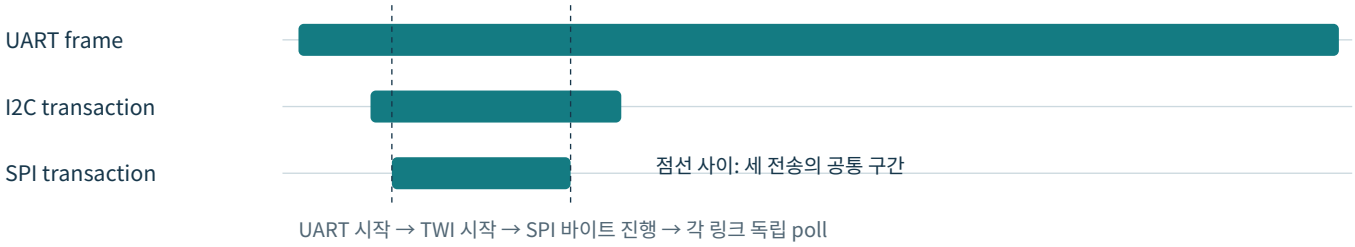
한 전송의 완료를 기다리지 않고 세 엔진을 번갈아 진행하여 실제 바이트 전송을 겹치게 한다.

엔진	초기 설정·처리 방식
software UART	Timer1 /8, OCR1A=207: 약 104 µs tick, 약 9615 baud. D3 직접 PORTD TX; D2 INT0 시작 검출 후 compare B로 중앙 RX sampling 검증.
I2C / TWI [6]	16 MHz에서 TWBR=72, TWPS=0: 100 kHz. start/address/data/stop을 짧은 ISR·상태 머신으로 진행하고 loop에 복귀.
SPI [5]	SPCR: SPE·MSTR, SPR1:SPR0=11, SPI2X=0: /128 = 125 kHz. SPIF poll로 한 바이트씩 진행하고 slave 준비 간격 확보.

하드웨어 SPI는 최저 125 kHz이다 [5]. 메인에서 blocking Wire.endTransmission이나 전체 프레임 blocking SPI transfer에 기대지 않는다. Timer1은 전용으로 예약하여 Servo 등과 충돌을 피한다. RX 위상·지터는 파형 검증 후 확정한다.

한 job 안에서 세 전송을 겹치게 시작

계획 파형 개념도 · 실측 아님 · 같은 job ID의 실제 전송



UART 32 B frame은 약 33.3 ms, SPI 32 B는 125 kHz에서 약 2.05 ms, I2C 32 B는 100 kHz에서 ACK 포함 약 3 ms와 주소·start/stop 시간이다. 모두 이상적인 전송 시간만의 추정이며 바이트 간격·ISR·계산·재시도가 추가된다. ISR 지연 10 µs 미만은 측정 목표이며 보장값이 아니다.

단계별 구현 순서와 저장할 증거

단계	실행·통과 기준	산출물
A → B	48뉴런 기준 동결 → 개별 32 B ACK/CRC → 실제 문맥의 3중 전송	model ID·패킷 로그·trace
C → D	host 50문맥 분할 비교 → 각 16뉴런 업로드 → 실물 부분 로짓 합산	50context.csv·선택 토큰
E → F	CRC/누락/지연/model mismatch/긴 입력 → 1대와 4대 같은48 비교	오류 복구·p50/p95·4빌드·stack watermark
G	동등성·동시성 통과 후에만 H576 재학습·분할 확장	품질·메모리·지연·최종 모델

ping 대신 실제 LOAD_CONTEXT/결과 청크의 공통 중첩을 확인한다. 디코딩 trace CSV/VCD·스크린샷, 4대 빌드·스택 계측을 보관한다. PC는 학습·검증·업로드·입력/표시만 담당한다.

검증 계획과 비교 지표

수치 동등성과 안전성

시험	목표·통과 기준
동일 작은 모델	고정 50문맥에서 로짓 abs/rel 1e-4 이내, 선택 토큰 일치
연속 생성	30/100문자 조건의 출력, 종료·길이 제한·반복률 기록
입력 복구	잘못된 입력·긴 줄을 처리하고 다음 작업 정상 수행
통신 오류	CRC 오류·stale job·워커 누락에서 명시적 오류·timeout
동시성	동일 추론 job에서 세 프로토콜의 실제 신호 중첩
메모리	4대 모두 빌드 크기와 런타임 high-water, 여유 ≥512 B 목표

100문자 조건은 현재 최대 80토큰 설정을 확장한 시험용 펌웨어로 평가한다. 512 B 여유는 잠정 목표이며 실제 최대 스택 계측을 함께 확인한다. 최종 결과에는 사용한 펌웨어·가중치·코퍼스 버전, 입력 및 trace를 저장한다.

결과 표의 틀: 아직 미측정

지표	1보드 작은 모델	4보드 같은 모델	4보드 확장 모델
Flash / SRAM	빌드 8614 / 462 B	실측 예정	실측 예정
문자당·전체 지연 p50/p95	실측 예정	실측 예정	실측 예정
고정 입력의 토큰 일치	호스트 검증 있음	검증 예정	별도 품질 평가
동시 전송 파형	해당 없음	필수 확보	필수 확보

UART만 보아도 입력 16 B + 출력 116 B = 132 B이며, 9600 baud 8N1에서 약 0.1375초의 전송 하한이 있다. 헤더·ACK·계산·다른 버스 비용은 추가된다. 이는 실측 지연이나 속도 개선의 약속이 아니다.

품질 평가와 주요 위험

학습에 쓰지 않은 프롬프트·문장을 분리하고 정상 종료율, 반복률, 유효 단어율 및 짧은 문장 자연스러움을 기록한다. 코퍼스가 작아 암기된 문장만 재현할 위험이 있으므로 학습 정확도와 미학습 입력 품질을 구분한다.

주요 위험은 UART 타이밍과 인터럽트 충돌, SPI slave의 ISR 응답 지연, I2C 청크 손실, 워커별 float 합산 차이, 확장에 따른 Flash·속도 부족이다. 링크 속도를 보수적으로 시작하고 작업 단위 재시도·명시적 실패를 구현한다. 확장보다 작은 모델의 동등성과 동시 통신을 우선한다.

10월 20일 발표까지의 실행 계획

일정과 단계별 산출물

기간	작업	단계 통과 산출물
10/7~9	1보드 실물 확인, 작은 모델·데이터 고정	기준 출력·빌드·입력 목록
10/10~12	UART·I2C·SPI 세 링크 개별 구현	패킷 왕복·CRC·timeout 로그
10/13~15	가중치 분할·로짓 합산·동시 전송	50문맥 비교·중첩 trace
10/16~18	오류·지연·메모리 시험, 확장 탐색	측정표·미학습 품질·최종 모델 선택
10/19	4분 40초 리허설·자료 정리	발표 영상·사진·trace·기여 표시 확인
10/20	5분 미만 발표	동작 시연·증거·제한점 설명

발표 스토리보드: 총 4분 40초

시간	내용	화면·증거
0:00~0:35	문제와 과제 요구	목표·세 프로토콜 조건
0:35~1:15	한 대의 신경망	구조·예시·측정 제한
1:15~2:10	네 대의 협력 계산	분할 수식·배선도
2:10~3:20	실물 데모와 동시 전송	짧은 생성·동일 job 파형
3:20~4:05	한 대/네 대 비교	실측 지연·메모리·품질 표
4:05~4:40	결론·제한·기여	달성 요건·한계·팀 역할

확장 모델이 불안정하면 검증된 작은 분산 모델을 사용한다. 발표용 영상은 현장 시연 실패에 대비한 증거로 준비하며, 세 프로토콜의 동시 전송 요건 자체를 생략하지 않는다.

참고 자료 및 로컬 근거

[1] Arduino Uno R3 공식 데이터시트: https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf

[2] HuggingFaceTB SmolLM2-135M 모델 카드: https://huggingface.co/HuggingFaceTB/SmolLM2-135M

[3] ArduinoCore-avr Wire.h (BUFFER_LENGTH): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/Wire/src/Wire.h

[4] ArduinoCore-avr SoftwareSerial.cpp (write/cli): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/SoftwareSerial/src/SoftwareSerial.cpp

[5] ArduinoCore-avr SPI.h (clock divider): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/SPI/src/SPI.h

[6] ArduinoCore-avr twi.c (TWBR 설정): https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/Wire/src/utility/twi.c

로컬 근거: README.md · VERIFICATION.md · model_metadata.json · corpus.txt · docs/model-selection.md. 현재 증거는 1보드 컴파일·호스트 검증이며, 4보드 구현 및 실물 측정은 앞으로 수행한다.